

Dance4Life: Plataforma de Sensorização para Envelhecimento Ativo através da Dança

Carlos da Mota Bergueira (pg11605@alunos.uminho.pt)
Diego Jefferson Mendes Silva (pg59999@alunos.uminho.pt)
Filipa Araújo Pereira (pg42201@alunos.uminho.pt)
Rui Manuel Martins Marques Rodrigues (pg7942@alunos.uminho.pt)

Universidade do Minho, Braga, Portugal

Abstract. Este artigo apresenta o Dance4Life, uma plataforma de sensorização e inteligência artificial desenvolvida no âmbito da unidade curricular de Sensorização e Ambiente do Mestrado em Inteligência Artificial. O sistema tem como objetivo promover o envelhecimento ativo através da dança, integrando dados meteorológicos (API OpenWeatherMap) com dados fisiológicos, inerciais e de localização recolhidos em dispositivos Android (smartphone e wearables). A arquitetura assenta em três pilares: um servidor Flask para ingestão e encaminhamento de dados, uma camada de agentes para processamento e persistência, e uma camada de visualização interativa com PowerBI e D3.js/Leaflet para análise exploratória. Em paralelo, os dados suportam o desenvolvimento de um agente de Aprendizagem por Reforço, treinado em ambiente simulado estocástico (*MovementEnv*) com algoritmos DQN e PPO, para otimização de intervenções personalizadas. O trabalho demonstra a viabilidade de uma plataforma distribuída, centrada no utilizador sénior, que combina contexto ambiental, segurança fisiológica e motivação social através da música e da dança.

Keywords: Sensorização · Dance4Life · Envelhecimento Ativo · Machine Learning · HAR

1 Introdução

O envelhecimento da população europeia constitui um dos maiores desafios sociais e de saúde pública do século XXI. Em Portugal, estima-se que, em 2050, mais de 35% da população terá mais de 65 anos, tornando imperativa a adoção de soluções tecnológicas que promovam a autonomia, a atividade física e a integração social da população sénior. O Dance4Life surge como resposta a este desafio, propondo uma plataforma inteligente que combina sensorização ambiental e fisiológica com mecanismos de ludificação baseados na dança. O sistema é concebido para funcionar em dispositivos móveis (smartphones e smartwatches), processando dados localmente segundo o paradigma de Edge Computing, em conformidade com os princípios de Privacy by Design exigidos pelo RGPD. Este

relatório documenta a segunda fase do trabalho prático, focada na implementação da plataforma: a definição e implementação dos coletores de dados, o tratamento e visualização dos dados recolhidos, e a conceção do agente de Aprendizagem por Reforço. Complementarmente, apresenta-se uma proposta de modelo de negócio que enquadra a viabilidade da solução em contextos reais. O documento está organizado da seguinte forma: a Secção 2 descreve a arquitetura global do sistema e os domínios abordados; a Secção 3 detalha os sensores e os coletores de dados implementados; a Secção 4 apresenta a exploração e visualização dos dados; a Secção 5 descreve o agente de Aprendizagem por Reforço concebido; a Secção 6 analisa resultados e limitações; e a Secção 7 apresenta o modelo de negócio.

2 Domínios, Objetivos e Arquitetura

2.1 Domínios Abordados

O Dance4Life opera na intersecção de três domínios principais:

Saúde e Bem-Estar Sénior Monitorização da atividade física, frequência cardíaca e padrões de movimento, com o objetivo de promover o exercício seguro e personalizado.

Ambiente Urbano Inteligente Integração de dados contextuais do ambiente, com foco nas condições meteorológicas e de localização.

Interação Social e Ludificação Matching de utilizadores com perfis compatíveis (Dancing Match) e estímulos musicais adaptativos que transformam a monitorização passiva numa experiência ativa e motivadora.

2.2 Objetivos

Os objetivos da fase prática organizam-se em três níveis:

1. **Sensorização e recolha:** Implementar coletores de dados para sensores físicos e virtuais, integrando a API OpenWeatherMap e os sensores inerciais, fisiológicos e de localização de dispositivos Android (smartphone e Wear OS).
2. **Análise e visualização:** Explorar, tratar e visualizar os dados recolhidos, aplicando técnicas de visualização interativa em D3.js e Leaflet para identificação de padrões temporais, geográficos e de participação social.
3. **Decisão inteligente:** Conceber e implementar um agente de Aprendizagem por Reforço para otimização da política de intervenção do sistema Dance4Life, equilibrando a promoção da atividade física com a prevenção de fadiga de alarme.

2.3 Arquitetura Global do Sistema

A arquitetura do Dance4Life foi concebida segundo o paradigma distribuído *edge-cloud*, procurando equilibrar três requisitos fundamentais e frequentemente conflitantes em sistemas de saúde digital: a latência de resposta às intervenções, a preservação da privacidade dos dados biométricos e a escalabilidade do processamento analítico. A separação de responsabilidades entre dispositivo e servidor não é, por isso, meramente técnica — reflecte uma decisão de desenho fundamentada nos princípios de *Privacy by Design* exigidos pelo RGPD, ao garantir que os sinais fisiológicos brutos são processados localmente e apenas métricas derivadas atravessam a rede. A arquitetura resultante, ilustrada na Figura 1, organiza-se em cinco camadas funcionais com interfaces bem definidas.

A camada de **aquisição no edge** é materializada em duas aplicações nativas para Android — uma para smartphone e outra para Wear OS — responsáveis pela captura contínua dos sinais inerciais (acelerómetro triaxial e giroscópio, via `TYPE_ACCELEROMETER` e `TYPE_GYROSCOPE`) e fisiológicos (frequência cardíaca via `TYPE_HEART_RATE`, com mecanismo de *fallback* baseado em estimativa por magnitudes inerciais quando o sensor não está disponível), bem como da localização geográfica enriquecida semanticamente por *reverse geocoding* (cidade, país e rua). Ambos os dispositivos partilham os mesmos contratos de abstração definidos no módulo `core` — as interfaces `SensorProvider`, `LocationProvider` e `DeviceProvider` — tornando o código de domínio independente da plataforma física e facilitando a extensão a outros tipos de dispositivos.

O **pré-processamento local** é executado pelo `DanceController`, que agrega os eventos sensoriais em janelas deslizantes, computa as magnitudes euclidianas dos vetores inerciais, estima o indicador de ritmo de movimento e constrói a observação compacta `MovementObservation` — com as variáveis *activityLevel*, *physicalFatigue* e *irritationLevel* — que alimenta a política de Aprendizagem por Reforço inferida localmente via ONNX Runtime. A periodicidade de envio ao servidor está fixada em 30 segundos, assegurando granularidade temporal suficiente para análise longitudinal sem sobrecarregar a rede.

A camada de **ingestão e serviços de contexto**, implementada em Flask, expõe uma API REST com cinco endpoints principais: `/collect_data_activities` para receção de telemetria de atividade; `/get_environment_data` para consulta de contexto meteorológico em tempo real junto da API OpenWeatherMap; `/set_movement_recommendation` para registo das recomendações geradas no edge; e `/get_user_match` e `/set_invite_status` para gestão do ciclo de vida dos convites sociais. O servidor mantém ainda um registo de agentes ativos com mecanismo de *heartbeat*, suportando a coordenação assíncrona da camada multiagente.

A **orquestração multiagente**, construída sobre a framework SPADE, implementa um pipeline de mensagens orientadas por ontologias e performativas, seguindo a ordem `SensorAgent` → `CoordinatorAgent` → `HarAgent` → `EnvironmentAgent` → `DatabaseAgent` para o fluxo principal de atividade. O *EnvironmentAgent* enriquece o payload com a temperatura ambiente obtida em tempo real, enquanto o *DatabaseAgent* persiste o evento enriquecido no Firebase.

Em paralelo, um ramo independente do *CoordinatorAgent* encaminha dados de atividade para o *MatchingAgent*, que aplica um modelo de agrupamento por níveis de atividade para gerar convites sociais entre utilizadores compatíveis. Esta arquitetura orientada a mensagens confere elevada modularidade e permite evoluir componentes individuais sem afetar o restante pipeline.

Por fim, a **camada de persistência e analítica** utiliza o Firebase como *backend* documental para armazenamento de eventos de atividade, convites, estados de *matching* e recomendações de movimento. Os dados persistidos são consumidos por dashboards analíticos implementados em D3.js e Leaflet, disponibilizando visualizações interativas de séries temporais, distribuição geográfica, rankings de participação e decomposição de scores por utilizador.

Esta arquitetura em cinco camadas reflecte um equilíbrio deliberado entre a capacidade de resposta em tempo real — garantida pelo processamento e inferência no edge — e a riqueza analítica proporcionada pela persistência centralizada. A adoção do paradigma multiagente com SPADE não se limita a uma conveniência de implementação: decorre da necessidade de coordenar comportamentos heterogéneos (reconhecimento de atividade, enriquecimento contextual, *matching* social e persistência) de forma assíncrona, auditável e com separação clara de responsabilidades. Do ponto de vista da privacidade, o facto de os sinais biométricos brutos nunca abandonarem o dispositivo — sendo apenas transmitidas métricas derivadas com granularidade de 30 em 30 segundos — concretiza operacionalmente o princípio de minimização de dados preconizado pelo RGPD, constituindo simultaneamente uma vantagem técnica (redução de largura de banda e latência) e uma garantia de conformidade regulatória.

2.4 Tecnologias e Ferramentas

A Tabela 1 resume o conjunto de tecnologias e bibliotecas que suportam a implementação do Dance4Life, organizadas por camada funcional. A selecção de cada ferramenta foi motivada por critérios de maturidade, adequação ao domínio e compatibilidade com os requisitos de privacidade e desempenho do sistema.

3 Sensores, Coletores de Dados e Ambiente

A recolha de dados no Dance4Life assenta numa arquitectura de sensorização heterogénea, que combina **sensores físicos** — transdutores embarcados nos dispositivos que convertem grandezas físicas em sinais digitais — e **sensores virtuais** — fontes cujos valores resultam de processamento sobre múltiplas fontes primárias ou da consulta a serviços externos [1]. Esta distinção estrutura a *pipeline* de aquisição do sistema: determina o local de processamento, o padrão de latência e as implicações de privacidade de cada sinal adquirido.

3.1 Sensores Físicos no Edge

Os sensores físicos operam em dois dispositivos Android — um smartphone e um smartwatch com Wear OS — acedidos através da **SensorManager** API com as

Table 1. Tecnologias e ferramentas utilizadas no Dance4Life, por camada funcional.

Camada	Tecnologia	Papel no sistema
Edge — Smart-phone e Wear OS	Kotlin + Android SDK	Aquisição de sensores, pré-processamento local e comunicação com o servidor
Edge — Inferência	ONNX Runtime (Android)	Execução da política de AR exportada em formato ONNX diretamente no dispositivo
Backend — In-gestão	Python 3 + Flask	Servidor REST para receção de telemetria, contexto ambiental e gestão de convites sociais
Backend — Multia-gente	SPADE 3	Orquestração assíncrona de agentes por troca de mensagens com ontologias e performativas FIPA
Persistência	Firebase Firestore	Base de dados documental para eventos de atividade, convites, <i>matching</i> e recomendações
Contexto ambiental	OpenWeatherMap API	Sensor virtual meteorológico (temperatura, humidade, vento) consumido pelo servidor
Aprendizagem por Reforço	Gymnasium + Stable-Baselines3	Definição do ambiente MovementEnv e treino comparativo de DQN e PPO
Visualização	D3.js + Leaflet	Dashboards analíticos interativos com séries temporais, mapas e rankings de participação

constantes `TYPE_ACCELEROMETER`, `TYPE_GYROSCOPE` e `TYPE_HEART_RATE`. Ambos os dispositivos partilham os contratos de abstracção definidos no módulo `core` (interfaces `SensorProvider`, `LocationProvider` e `DeviceProvider`), tornando o código de domínio independente da plataforma física subjacente.

O **acelerómetro triaxial** é um transdutor inercial de tecnologia MEMS (*Micro-Electro-Mechanical Systems*) que mede a aceleração linear nos três eixos cartesianos $\{x, y, z\}$ em m/s^2 . Por incorporar a componente gravítica, o sinal capturado reflecte simultaneamente o movimento voluntário do utilizador e a orientação do dispositivo, sendo o sinal primário para reconhecimento de actividade humana (HAR) [4]. O **giroscópio triaxial**, igualmente de base MEMS, mede a velocidade angular por eixo em rad/s , explorando o efeito de Coriolis sobre estruturas vibrantes de escala microscópica. Enquanto o acelerómetro captura forças de translação, o giroscópio captura a dinâmica rotacional; a fusão das magnitudes euclidianas dos dois vectores — computada pelo **DanceController** — caracteriza o padrão de movimento tridimensional de forma mais robusta do que qualquer dos sensores isoladamente [7].

O **sensor de frequência cardíaca** (FC), baseado em fotopletismografia óptica (PPG) no smartwatch, fornece a frequência cardíaca instantânea em batimentos por minuto (bpm). Dado que este sensor pode tornar-se indisponível por ausência de contacto dérmico ou limitação de hardware, o sistema implementa um mecanismo de *fallback*: o valor é estimado a partir das magnitudes inerciais, assegurando a completude dos *payloads* e a continuidade do cálculo da variável *physicalFatigue* no **MovementObservation**. A **localização geográfica** é obtida por fusão de GPS e rede celular (AGPS) via `LocationProvider`, enriquecida por *reverse geocoding* com o **Android Geocoder**, que converte coordenadas

em metadados textuais (cidade, país e rua). A aplicação agrega estes sinais em *payloads* periódicos (*ritmo*, *acc*, *gyro*, *hr*, coordenadas e cidade), transmitidos ao servidor com intervalo fixo de 30 segundos.

3.2 Sensor Virtual Meteorológico

No contexto da sensorização heterogénea, um **sensor virtual** é uma abstracção que produz uma grandeza observável a partir de fontes primárias sem que exista um transdutor dedicado para essa grandeza [1]. O contexto meteorológico constitui o exemplo central desta categoria no Dance4Life: temperatura exterior, humidade relativa e velocidade do vento não podem ser medidas com fiabilidade por sensores embarcados — cujas leituras de temperatura são dominadas pela dissipação térmica do processador — sendo por isso obtidas via API REST da OpenWeatherMap (`/data/2.5/weather`). Esta decisão de desenho é deliberada: um termómetro embarcado mediria essencialmente o calor interno do *chipset*, e não a temperatura do ambiente em que o utilizador se encontra, variável com relevância directa para a modulação de recomendações de dança em contexto exterior.

O módulo `external_sensors.py` abstrai este sensor virtual, suportando consulta por coordenadas geográficas ou por cidade, com devolução de variáveis em unidades métricas (`units=metric`). O servidor Flask integra o dado ambiental através do endpoint `/get_environment_data`, tornando temperatura e humidade disponíveis tanto para a camada de agentes SPADE (*EnvironmentAgent*) como para o espaço de observação do agente de Aprendizagem por Reforço.

3.3 Inventário de Sensores e Variáveis Observáveis

A Tabela 2 sintetiza os sensores e fontes de dados efetivamente integrados no protótipo, distinguindo aquisição física no edge e aquisição virtual de contexto. Esta explicitação é relevante para reprodutibilidade experimental e para delimitação do espaço observável do sistema.

No edge, os dados inerciais e fisiológicos são adquiridos de forma assíncrona e fundidos localmente. Quando a leitura de frequência cardíaca não está disponível, o sistema aplica uma estimativa robusta baseada nas magnitudes de acelerómetro e giroscópio, assegurando continuidade operacional. A localização é enriquecida semanticamente por *reverse geocoding*, permitindo análises por cidade e contexto urbano.

3.4 Servidor de Recolha de Dados (Flask)

O servidor Flask constitui o ponto de entrada de dados e de coordenação de serviços. Na implementação atual, expõe, entre outros, os endpoints `/collect_data_activities`, `/get_environment_data`, `/set_movement_recommendation`, `/get_user_match` e `/set_invite_status`. Este desenho permite separar claramente três responsabilidades: ingestão de telemetria, disponibilização de contexto ambiental e gestão de interação social/recomendação.

Table 2. Sensores e fontes de dados implementados no Dance4Life.

Origem	Sensor/Fonte	Variáveis principais	Unidade/Formato
Smartphone e Wear OS	Acelerômetro triaxial	acc_x , acc_y , acc_z , magnitude inercial	m/s ² (e magnitude derivada)
Smartphone e Wear OS	Giroscópio triaxial	$gyro_x$, $gyro_y$, $gyro_z$, magnitude angular	rad/s (e magnitude derivada)
Smartphone e Wear OS	Sensor de frequência cardíaca	frequência cardíaca instantânea (com fallback estimado)	bpm
Smartphone e Wear OS	Localização (GPS/rede + Geocoder)	latitude, longitude, cidade, país, rua	coordenadas + metadados textuais
Servidor (sensor virtual)	OpenWeatherMap API	temperatura, humidade, velocidade do vento, timestamp meteorológico	°C, %, m/s, Unix time

Na cadeia de ingestão, cada payload recebido é enriquecido com metadados temporais e encaminhado para a camada de agentes SPADE. O fluxo principal de atividade segue a ordem *SensorAgent* → *CoordinatorAgent* → *HarAgent* → *EnvironmentAgent* → *DatabaseAgent*, onde o *EnvironmentAgent* acrescenta contexto meteorológico e o *DatabaseAgent* realiza persistência em Firebase. Em paralelo, o *MatchingAgent* processa progressão por clusters e gera convites sociais, preservando separação entre decisão social e persistência. Esta arquitetura orientada a mensagens aumenta modularidade, facilita evolução independente dos agentes e suporta rastreabilidade do processamento ponta-a-ponta.

4 Exploração e Visualização dos Dados

A camada de visualização do Dance4Life materializa o fluxo completo de dados sensoriais: os sinais capturados nos dispositivos Android percorrem o pipeline de ingestão, são enriquecidos e persistidos no Firebase Firestore, e tornam-se consultáveis pelos dashboards analíticos implementados em D3.js e Leaflet. Esta secção descreve esse percurso desde a aquisição sensorial até à representação visual.

4.1 Da Captura Sensorial ao Dado Persistido

Cada *payload* transmitido ao servidor a cada 30 segundos transporta variáveis com origem direta nos sensores físicos e virtuais descritos na Secção 3: o **ritmo** (magnitude de movimento inercial derivada do acelerômetro e giroscópio), a **frequência cardíaca** (PPG ou estimativa inercial por *fallback*), as **coordenadas geográficas e cidade** (GPS/AGPS + *reverse geocoding*), e os **dados meteorológicos** (temperatura, humidade e vento, via OpenWeatherMap). No servidor Flask, cada *payload* é validado estruturalmente (campos obrigatórios e tipos de dados), enriquecido com metadados temporais e encaminhado para o pipeline

Table 3. Visualizações implementadas e respetivas origens sensoriais.

Visualização	Variáveis sensoriais expostas
Ranking Top 10 (barras)	Score agregado por utilizador: atividade (inercial), convites aceites, <i>matching</i> (ritmo + FC) e recomendações
Série temporal mensal	Volume de registos de atividade ao longo do tempo; reflete a periodicidade de emissão de <i>payloads</i> pelos dispositivos
Mapa por cidade (Leaflet)	Coordenadas GPS e cidade por <i>reverse geocoding</i> ; intensidade de atividade inercial por localização
Distribuição de convites	Estado dos convites sociais gerados pelo ClusteringAgent a partir do <i>ritmo</i> inercial
Cartão individual (circular)	Decomposição do score por componente: atividade inercial, FC, participação social e recomendações

multiagente SPADE, onde o *EnvironmentAgent* acrescenta o contexto meteorológico em tempo real antes de o *DatabaseAgent* persistir o evento enriquecido no Firebase. É este registo final — contendo variáveis inerciais, fisiológicas, geográficas e ambientais numa única entrada temporal — que alimenta a camada de análise.

4.2 Análise Exploratória

Os dados persistidos foram explorados com o objetivo de identificar padrões comportamentais de utilização da plataforma a partir das variáveis sensoriais. A distribuição de **atividade por utilizador** (derivada das magnitudes inerciais do acelerómetro e giroscópio) permitiu distinguir perfis de maior e menor participação. A **análise temporal mensal** evidenciou variações no volume de recolha ao longo do período observado, refletindo a periodicidade real da emissão de *payloads* pelos dispositivos. A **análise geográfica por cidade**, suportada por visualização georreferenciada com coordenadas GPS enriquecidas por *reverse geocoding*, permitiu identificar concentração de atividade em determinados contextos urbanos. A **distribuição de estados de convite** (aceite, recusado, pendente) e os indicadores de *matching* social forneceram uma perspetiva complementar sobre o envolvimento social promovido pelo sistema com base nos perfis de ritmo e frequência cardíaca dos utilizadores.

4.3 Visualizações Implementadas e Variáveis Sensoriais Expostas

As visualizações foram implementadas com D3.js e Leaflet, escolhidos pela sua flexibilidade na renderização interativa de séries temporais e dados geoespaciais. A Tabela 3 resume cada visualização e as variáveis sensoriais que expõe.

Table 4. Variáveis de estado do MDP e respetivas origens sensoriais.

Variável	Intervalo	Origem sensorial
Nível de Atividade	[0, 10]	Magnitude euclidiana do acelerómetro (<code>TYPE_ACCELEROMETER</code>) e giroscópio (<code>TYPE_GYROSCOPE</code>), fundada pelo <code>DanceController</code>
Fadiga Física	[0, 10]	Frequência cardíaca (<code>TYPE_HEART_RATE</code> , PPG); em caso de indisponibilidade, estimativa por <i>fallback</i> inercial
Irritação (Fadiga de Alarme)	[0, 10]	Frequência de intervenções emitidas pelo sistema ao longo da sessão

5 Aprendizagem Automática: Intervenção e Agrupamento Social

O Dance4Life incorpora dois modelos de aprendizagem automática complementares que consomem diretamente os dados produzidos pela camada de sensorização descrita na Secção 3: um agente de Aprendizagem por Reforço (AR) para otimização das intervenções individuais, e um modelo de agrupamento (*clustering*) para emparelhamento social entre utilizadores (*Dancing Match*) [3,2].

5.1 Formulação MDP e Origens Sensoriais do Estado

O problema de intervenção é formalizado como um Processo de Decisão de Markov (MDP). O espaço de estado é contínuo, com três variáveis em [0,10] construídas pelo `DanceController` a partir dos sinais sensoriais recolhidos nos dispositivos Android, conforme a Tabela 4.

O espaço de ação é discreto com quatro níveis: **Silêncio** (sem intervenção), **Baixa**, **Média** e **Alta Intensidade**. A função de recompensa codifica os objetivos clínicos: +15 (contínua) quando o utilizador se encontra na zona ideal (atividade $\in [4, 7]$, fadiga < 7 , irritação < 5); -5 (contínua) em sedentarismo perigoso (atividade < 2); e -50 (terminal) em falha por exaustão fisiológica ou abandono por fadiga de alarme.

5.2 Ambiente, Algoritmos e Resultados

O agente é treinado no ambiente simulado estocástico `MovementEnv` (Gymnasium/Farama Foundation), que imita a dinâmica fisiológica e comportamental de um utilizador sénior com base nos mesmos três sinais derivados dos sensores. Foram implementados e comparados dois algoritmos via Stable-Baselines3: **DQN** (*value-based*, com *Experience Replay* e *Target Network*) e **PPO** (*policy-based/actor-critic*, com função objetivo *clipped surrogate*). Ambos convergiram para *survival rate* de 100% em 30 episódios de avaliação (Tabela 5); o DQN atingiu este desempenho ao 20 000 timestep, enquanto o PPO requereu 260 000 timesteps.

Table 5. Comparação de desempenho DQN vs. PPO no ambiente `MovementEnv`.

Métrica	DQN	PPO
Recompensa Média (30 episódios)	1437.50	1436.00
Duração Média de Episódio (passos)	96.00	96.00
Survival Rate	100.00%	100.00%
Timesteps até convergência	20 000	260 000

5.3 Agrupamento e Matching Social (`ClusteringAgent`)

Em paralelo ao agente AR, o `ClusteringAgent` implementa um modelo de agrupamento de utilizadores baseado nos mesmos dados sensoriais, com o objetivo de gerar convites sociais entre utilizadores com perfis de atividade compatíveis — o *Dancing Match*.

Os utilizadores são classificados em quatro *clusters* de progressão determinados pelo valor de *ritmo* — a magnitude de movimento inercial derivada da fusão do acelerómetro e giroscópio pelo `DanceController`: **Iniciante** ($ritmo < 17$), **Moderado** ($17 \leq ritmo < 18.5$), **Avançado** ($18.5 \leq ritmo < 20$) e **Expert** ($ritmo \geq 20$). Para efeitos de emparelhamento entre utilizadores, aplica-se um algoritmo K-Means em memória (máximo 20 iterações), sem dependência de bibliotecas externas, sobre vetores de cinco dimensões: coordenadas geográficas normalizadas, frequência cardíaca normalizada, *ritmo* e género musical. O *score* de compatibilidade entre dois utilizadores é calculado como combinação ponderada de três critérios: semelhança musical (45%), proximidade geográfica (35%) e semelhança de sinais vitais — FC e *ritmo* — (20%). São gerados convites apenas para pares com $score \geq 0.45$ e distância ≤ 8 km. Em simultâneo, o modelo emite um convite de progressão individual, motivando o utilizador a atingir o *cluster* seguinte com base no valor atual do seu *ritmo* inercial.

6 Resultados, Limitações e Recomendações

6.1 Sumário dos Resultados Obtidos

A implementação da plataforma `Dance4Life` na sua fase prática permitiu alcançar os seguintes resultados:

- Integração do sensor virtual meteorológico (`OpenWeatherMap`), com recolha contínua de dados ambientais para suporte contextual.
- Implementação de um simulador de dispositivo Android funcional, capaz de gerar e transmitir dados fisiológicos e inerciais realistas para o servidor Flask com intervalo de 30 segundos.
- Desenvolvimento de um servidor Flask robusto, com mecanismo de fusão sensorial na ingestão e persistência em base de dados documental (`Firestore`).
- Criação de um conjunto de visualizações `PowerBI` e `D3.js/Leaflet` interativas que permitem a análise exploratória dos dados recolhidos, incluindo ranking, série temporal, distribuição geográfica e estado de convites.

- Conceção e avaliação comparativa de dois agentes de Aprendizagem por Reforço (DQN e PPO) num ambiente simulado estocástico (**MovementEnv**), com formulação MDP documentada e métricas de avaliação definidas.

6.2 Análise Crítica e Limitações

A principal limitação da implementação atual reside na utilização exclusiva de dados simulados tanto para os sensores inerciais e fisiológicos como para o ambiente de treino do agente RL (**MovementEnv**). Os dados gerados seguem distribuições estocásticas parametrizadas que, embora mais realistas do que distribuições uniformes simples, não capturam a plena complexidade e variabilidade real dos padrões fisiológicos e comportamentais de utilizadores seniores. Esta limitação condiciona a capacidade de generalização da política aprendida pelo agente para contextos do mundo real. Uma limitação adicional reside na cobertura ambiental ainda centrada em variáveis meteorológicas, o que reduz a representatividade de contexto face a cenários urbanos mais complexos. Do ponto de vista do agente RL, a estocasticidade do ambiente **MovementEnv** introduz variância nas estimativas de recompensa durante o treino. Na avaliação comparativa realizada (30 episódios), ambos os modelos apresentaram survival rate de 100% e duração média de 96 passos por episódio; o DQN obteve recompensa média ligeiramente superior, enquanto o PPO mostrou comportamento igualmente estável. Estes resultados sugerem desempenho robusto dos dois métodos no ambiente simulado, carecendo ainda de validação com dados reais.

6.3 Recomendações para Trabalho Futuro

Com base na análise crítica realizada, identificam-se as seguintes recomendações para o desenvolvimento futuro da plataforma:

- Validação do agente RL com utilizadores reais ou com dados recolhidos de voluntários, substituindo gradualmente o **MovementEnv** por um ambiente de simulação calibrado com dados empíricos reais de população sénior.
- Extensão do espaço de estado do MDP com variáveis contextuais adicionais, permitindo ao agente adaptar as intervenções ao contexto ambiental em tempo real de forma mais robusta.
- Implementação do paradigma de Federated Learning para personalização distribuída da política do agente, preservando a privacidade e adaptando as intervenções às características biomecânicas individuais de cada utilizador.
- Integração de um módulo de inferência implícita de preferências musicais, correlacionando os dados de acelerómetro com as músicas reproduzidas, para enriquecer o espaço de ação do agente com recomendações musicais personalizadas.
- Desenvolvimento de uma interface móvel nativa (Android/iOS) para substituição gradual do simulador e testes em condições reais de utilização.

7 Modelo de Negócio

7.1 Proposta de Valor e Cliente-Alvo

O Dance4Life posiciona-se como uma plataforma B2B2C (Business-to-Business-to-Consumer), cujo cliente primário são instituições de cuidado sénior – centros de dia, lares de idosos e programas municipais de envelhecimento ativo – e cujo utilizador final é a população sénior (65+ anos). A proposta de valor diferencia-se em três eixos: personalização baseada em dados (adaptação da atividade e da música ao perfil individual), segurança fisiológica (monitorização da frequência cardíaca e alertas em tempo real), e combate ao isolamento social (emparelhamento social – Dancing Match).

7.2 Parcerias Estratégicas

A viabilidade do Dance4Life assenta num ecossistema de parcerias estratégicas:

- Instituições de saúde e cuidado sénior: parceiros de distribuição e validação clínica (ex.: Santa Casa da Misericórdia, SCML, Misericórdias locais).
- Fabricantes de wearables: integração nativa com dispositivos como Fitbit, Garmin ou Apple Watch para acesso a dados fisiológicos reais de alta qualidade.
- Serviços de streaming musical: licenciamento de conteúdo musical adaptado (ex.: Spotify API, TIDAL) para alimentar o motor de recomendação musical.
- Municípios e programas de saúde pública: parceiros de financiamento no contexto de cidades inteligentes e políticas de envelhecimento ativo.

7.3 Estratégia de Monetização

A estratégia de monetização combina dois modelos complementares:

- Subscrição institucional (SaaS): licença mensal por instituição, com preço escalável em função do número de utilizadores ativos. Este modelo garante receita recorrente e previsível.
- Modelo freemium para utilizadores individuais: versão gratuita com funcionalidades básicas (monitorização de atividade e sugestões musicais), e versão premium com funcionalidades avançadas (emparelhamento social, relatórios de saúde detalhados, integração com wearables).

Adicionalmente, os dados anonimizados e agregados gerados pela plataforma constituem um ativo de valor para investigação académica e para entidades de saúde pública, abrindo uma terceira linha de receita através de parcerias de investigação e venda de insights analíticos, sempre em conformidade com o RGPD.

8 Conclusão

O presente trabalho documentou o desenvolvimento da fase prática do projeto Dance4Life, uma plataforma de sensorização e inteligência artificial para a promoção do envelhecimento ativo através da dança. A implementação demonstrou a viabilidade técnica da integração de dados meteorológicos (OpenWeatherMap) com dados de dispositivos Android (smartphone e wearables), num servidor Flask capaz de realizar ingestão, encaminhamento por agentes e persistência em tempo real. A exploração dos dados recolhidos através de visualizações PowerBI e D3.js/Leaflet revelou padrões temporais, geográficos e de participação social úteis para monitorização do sistema. O agente de Aprendizagem por Reforço concebido com formulação MDP e implementação comparativa de DQN e PPO constitui o motor de decisão inteligente da plataforma, capaz de aprender políticas de intervenção que equilibram atividade física e bem-estar psicológico. O Dance4Life demonstra como a convergência de sensorização ambiental, processamento inteligente e ludificação pode criar uma solução tecnológica diferenciada para um dos maiores desafios sociais contemporâneos: o envelhecimento ativo. A proposta de modelo de negócio delineada reforça a viabilidade da solução em contextos reais, abrindo caminho para uma implementação piloto em parceria com instituições de cuidado sénior. Como trabalho futuro, a prioridade recai na substituição dos dados simulados por dados reais de utilizadores seniores e na implementação do módulo de Aprendizagem Federada, que permitirá ao sistema aprender de forma distribuída e privacidade-preservante, completando a visão de um sistema verdadeiramente inteligente, adaptativo e centrado no utilizador.

References

1. Bouchabou, D., Nguyen, S.M., Lohr, C., LeDuc, B., Kanellos, I.: A Survey of Human Activity Recognition in Smart Homes Based on IoT Sensors Algorithms. *Sensors* **21**(18), 6037 (2021).
2. Liu, C., Thiamwong, L., Nguyen, T., Suarez, J.R., Park, J.-H., Lou, Q., Xie, R.: A reinforcement learning approach to enhance physical activity and reduce fall risk in older adults. *Innovation in Aging* 8(Suppl. 1), 1081–1081 (2024). <https://doi.org/10.1093/geroni/igae098.3473>
3. Gönül, S., Naml, T., Coar, A., Toroslu, I.H.: A reinforcement learning based algorithm for personalization of digital, just-in-time, adaptive interventions. *Artificial Intelligence in Medicine* 115, 102062 (2021). <https://doi.org/10.1016/j.artmed.2021.102062>
4. Hossen, M.A., Abas, P.E.: Machine Learning for Human Activity Recognition: State-of-the-Art Techniques and Emerging Trends. *J. Imaging* 11(3), 91 (2025).
5. Liao, P. et al.: Personalized HeartSteps: A Reinforcement Learning Algorithm for Optimizing Physical Activity. *ACM* (2019). <https://doi.org/10.1145/3381007>
6. Lu, Y. et al.: StepCountJITAI: simulation environment for RL with application to physical activity adaptive intervention. *arXiv:2411.00336* (2024).
7. Paraschiakos, S. et al.: Activity recognition using wearable sensors for tracking the elderly. *User Model. User-Adapt. Interact.* **30**, 567–605 (2020).

8. Peng, C. et al.: Promoting physical activity in middle-aged and older adults through digital intervention via deep reinforcement learning. *Int. J. Human-Computer Interaction* (2024). <https://doi.org/10.1080/23311908.2025.2541720>
9. Gymnasium (Farama Foundation): Framework para ambientes de Aprendizagem por Reforço. <https://gymnasium.farama.org/> (acedido em maio de 2026).
10. Stable-Baselines3: Biblioteca de algoritmos de Deep RL em PyTorch. <https://stable-baselines3.readthedocs.io/> (acedido em maio de 2026).
11. OpenWeatherMap API Documentation. <https://openweathermap.org/api> (acedido em maio de 2026).

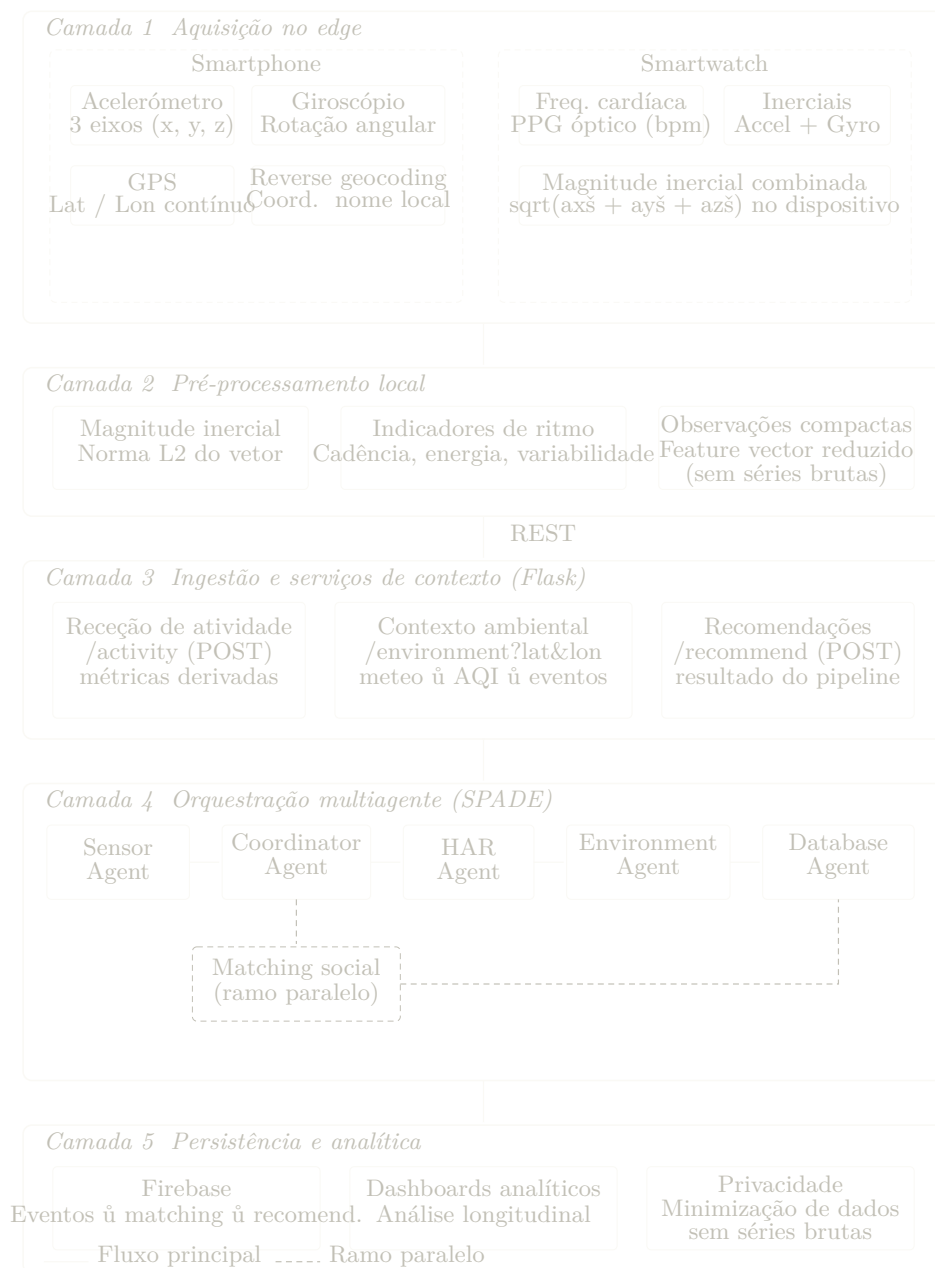


Fig. 1. Arquitetura global do sistema Dance4Life (visão em camadas e fluxo de dados).

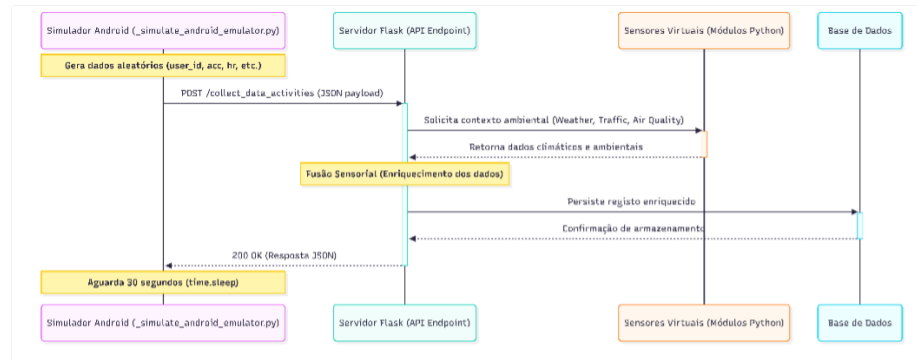


Fig. 2. Diagrama de sequência do fluxo de recolha de dados.

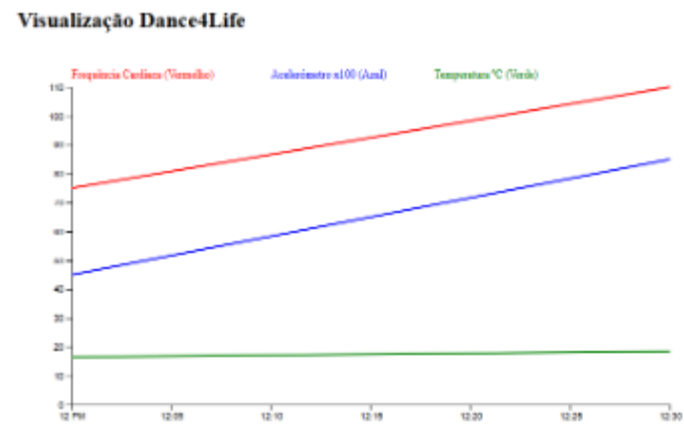


Fig. 3. Séries temporais das variáveis sensoriais recolhidas (D3.js).

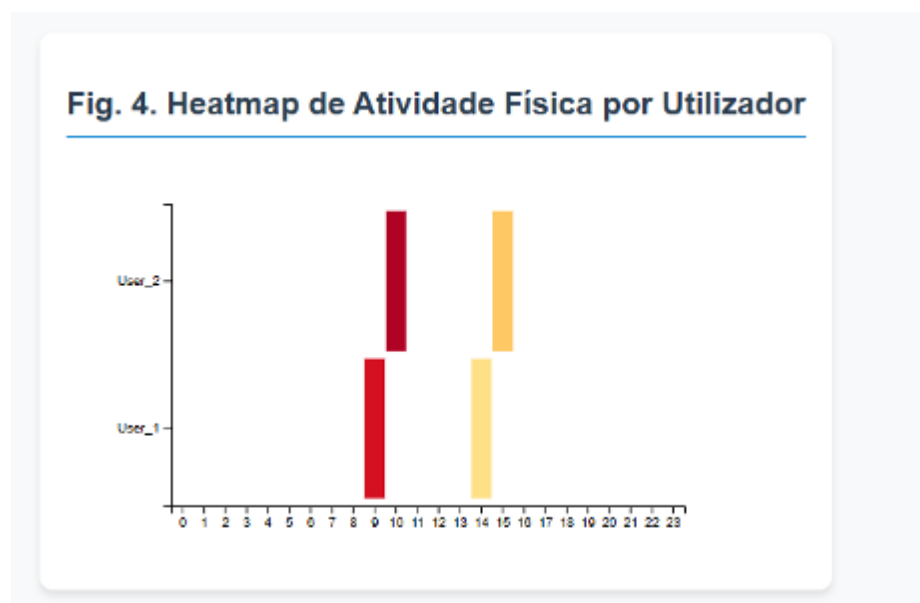


Fig. 4. Visualização de intensidade de atividade no dashboard (D3.js).